

2. b. Unrolled List

–Delete Element

–Time Complexity Analysis

Program:

```
void deleteElement(int val) {
    if (head == NULL) {
        cout << "The list is empty. Nothing to delete.\n";
        return;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {

                // Element found: Shift remaining elements left
                for (int j = i; j < temp->numElements - 1; j++) {
                    temp->elements[j] = temp->elements[j + 1];
                }
                temp->numElements--;

                cout << "Element " << val << " deleted
successfully.\n";

                // Check if the node is now completely empty
                if (temp->numElements == 0) {
                    if (prev == NULL) {
                        head = temp->next;
                        free(temp);
                    } else {
                        prev->next = temp->next;
                        free(temp);
                    }
                    cout << "An empty node was removed from the
list.\n";
                }
            }
        }
    }
}
```

```
        return;
    }
}
prev = temp;
temp = temp->next;
}

cout << "Element " << val << " not found in the list.\n";
}
....
    case 2:
        cout << "Enter integer value to delete: ";
        cin >> value;
        deleteElement(value);
        break;
```

Time Complexity Analysis

Variable Definitions

- *Let n = total number of elements stored across the entire unrolled linked list.*
- *Let N = total number of nodes in the list.*
- *Let B = capacity of each node ($MAX_{ELEMENTS} = 4$).*

Note: Because each node holds a fixed constant capacity of elements

($B = 4$), the number of nodes $N \approx \left\lceil \frac{n}{4} \right\rceil$, meaning $O(N) = O(n)$.

Say, if we have 13 elements in the list, then $n = 13$ and these total of n

elements grouped into 4 blocks i.e. $\frac{n}{4} = \frac{13}{4} = 3.25$. As we cannot allocate

fraction of a node, hence we will use ceiling function $\left\lceil \frac{n}{4} \right\rceil = \left\lceil \frac{13}{4} \right\rceil$

= $\lceil 3.25 \rceil = 4$. That is number of Nodes ' N ' = 4 to store these $n = 13$

elements.

i.e. $N = \left\lceil \frac{n}{B} \right\rceil$, where B is $MAX_{ELEMENTS} = 4$.

1. *Best Case Time Complexity: $O(1)$*

The best – case scenario occurs when the function performs the minimum possible number of operations and terminates immediately:

1. **[Absolute Best Case]** *Empty List: If the list is completely empty (`head == NULL`), the function checks the condition, prints an error message, and returns immediately.*

2. **[Not Absolute Best Case]** *Element at the Very Beginning:*
If the target element is located at index 0 of the first node (`head → elements[0] == val`):
 - *The outer **while** loop runs only 1 time.*
 - *The search **for** loop finds the match on the first iteration (**$i = 0$**).*
 - *The shifting **for** loop shifts at most **$B - 1 = 4 - 1 = 3$** elements to the left, which takes constant time ($O(1)$).*
 - *Node deallocation (if the node becomes empty) also takes constant time ($O(1)$).*

2. Worst Case Time Complexity: $O(n)$ (or $O(N)$)

The worst – case scenario occurs when the algorithm is forced to perform the maximum possible number of comparisons and pointer traversals:

1. **[Absolute Worst Case]Element Not Found:** *The element does not exist in the list. The function must inspect every element across every node before reaching `temp == NULL` and printing Element not found.*

2. **[Not Absolute Worst Case]Element at the Very End:**
The element is located in the last position of the last node in the list.

[Absolute Worst Case] Element Not Found:

- *Let n = total number of elements stored across the entire unrolled linked list.*
- *Let N = total number of nodes in the list.*
- *Let B = capacity of each node ($MAX_{ELEMENTS} = 4$).*

```
while (temp != NULL) {
```

Nodes Traversal: The outer loop visits every single node. $Cost = c_n \times N$.

- c_n = The cost of hopping from one node to the next.

```
for (int i = 0; i < temp->numElements; i++) {
    if (temp->elements[i] == val) {
```

Elements Compared: The inner loop runs for every element accross all nodes.

$Costs = c_s \times n$.

- c_s = The cost of checking one element (the if comparison.)

```
for (int j = i; j < temp->numElements - 1; j++) {
    temp->elements[j] = temp->elements[j + 1];
}
```

Here we see `` $j < temp \rightarrow numElements - 1$ `` i.e. $j < B - 1$, but because the element is never found, the shifting loop never executes. $Cost = c_{sh} \times 0 = 0$.

- c_{sh} = The cost of shifting one element in the array.

```
temp->numElements--;

cout << "Element " << val << " deleted
successfully.\n";

// Check if the node is now completely empty
if (temp->numElements == 0) {
    if (prev == NULL) {
        head = temp->next;
        free(temp);
    } else {
        prev->next = temp->next;
        free(temp);
    }
}
```

```

        cout << "An empty node was removed from the
list.\n";
    }

```

Deletions: Nothing is deleted. Hence decrement occurs. Cost = $c_d \times 0 = 0$.

- c_d = The constant cost of bookkeeping (decrementing numElements, freeing a node)

Why is this considered a constant cost(c_d)?

No matter how many total elements (n) or nodes (N) exist in our list,

this specific block of code always takes the exact same number of steps to run:

- It does 1 subtraction ($temp \rightarrow numElements --$).
- It evaluates at most 2 if conditions.
- It does 1 pointer assignment (either $head = \dots$ or $prev \rightarrow next = \dots$).
- It performs 1 memory deallocation ($free(temp)$).

Mathematical Equation: $T_1 = (c_n \times N) + (c_s \times n) + 0 + 0$

As, $N \leq n$, the dominant term is n .

1. In terms of elements : $O(n)$.

Now,

```

while (temp != NULL) {
    for (int i = 0; i < temp->numElements; i++) {
        if (temp->elements[i] == val) {

```

That is we traverse each node blocks : N

And each node block elements traversed from 0 to $B - 1$ i.e. $\sum_{i=0}^{B-1} 1$

And number of comparisons also: $\sum_{i=0}^{B-1} 1$ times.

$$\sum_{i=0}^{B-1} 1 = 1 + 1 + \dots = (\text{Top Bound} - \text{Bottom Bound}) + 1 = (B - 1 - 0) + 1 = B.$$

That is $N \times B$ comparisons .

Relationship to n : *The variable n represents the exact number of active elements currently stored across the entire data structure. Because N nodes can each hold a maximum of B elements, the absolute physical capacity of the unrolled linked list is $N \times B$ slots. This gives us the mathematical guarantee that $n \leq N \times B$.*

```
while (temp != NULL) {
    for (int i = 0; i < temp->numElements; i++) {
        if (temp->elements[i] == val) {

            // Element found: Shift remaining elements left
            for (int j = i; j < temp->numElements - 1; j++) {
                temp->elements[j] = temp->elements[j + 1];
            }
        }
    }
}
```

The local shift:

```
for (int j = i; j < temp->numElements - 1; j++) {
    temp->elements[j] = temp->elements[j + 1];
}
```

1. *At most N nodes are visited.*
2. *Each nodes contains B elements.*
3. *Thus there can be at most NB element comparisons.*
4. *Then at most ' $B - 1$ ' local shift occurs. i. e.*

$$\sum_{i=0}^{B-2} 1 = (B - 2 - 0) + 1 = B - 1 \text{ local shift.}$$

[As shift starts from $j = 0$ to $j < B - 1$ i. e. 0 to $B - 2$]

$$\begin{aligned} \text{Therefore, } O(NB + B) + O(-1) \\ = O(NB + B) \end{aligned}$$

As , during , Absolute Worst Case, when Element Not Found , +B shifting term is completely absent(0).

$$\begin{aligned} \text{Therefore , } O(NB + 0) \\ = O(NB) \end{aligned}$$

Note: N is the dominant term and B is just constant , here 4.

$$\begin{aligned} \therefore O(4N) \\ = O(N) \end{aligned}$$

2. Hence , In terms of nodes and blocks : $O(N)$.

$\therefore$ [Absolute Worst Case] Element Not Found $:= O(n)$ or $O(N)$.

[Not Absolute Worst Case]

Element at the Very End

1. **Nodes Traversed** : The outer loop visits every single node. Cost = $c_n \times N$.

2. **Elements Compared**: The inner loop checks every element until the very last one.
 $= c_s \times n$.

3. **Elements Shifted**: Because the element is at the very end of the node's internal array, there are no elements left to get shifted . The shifting loop condition $j < temp \rightarrow numElements - 1$ immediately evaluates to false.

Hence 0 elements are shifted . Cost = $c_{sh} \times 0 \text{ times} = 0$.

4. **Deletions:** The node bookkeeping executes, constant amount of time . Costs
 $= c_d \times 1 \text{ time} = c_d$.

$$\therefore T_2 = (c_n \times N) + (c_s \times n) + 0 + c_d$$

As , $N \leq n$, the dominant term is n .

1. In terms of elements : $O(n)$.

Also , as shifting term : $+B$ doesnot exist here , therefore:

$$\begin{aligned} \text{Therefore , } O(NB + 0) \\ = O(NB) \end{aligned}$$

Note: N is the dominant term and B is just constant , here 4.

$$\begin{aligned} \therefore O(4N) \\ = O(N) \end{aligned}$$

2. Hence , In terms of nodes and blocks : $O(N)$.

Hence , Worst Case Time Complexity : $O(n)$ or $O(N)$.

Average Case Time Complexity

A. Average Comparisons

A. Based on Loop

In a Node:

```
while (temp != NULL) {
```

Inner loop comparisons:

```
for (int i = 0; i < temp->numElements; i++) {  
    if (temp->elements[i] == val) {
```

- *When we want to delete the 1st element, the inner loop comparison runs 1 time.*

i. e. temp → elements[0] == val [True]

- *When we want to delete the 2nd element, the inner loop comparison runs 2 times.*

i. e. temp → elements[0] == val [False]

temp → elements[1] == val [True]

- *When we want to delete the 3rd element, the inner loop comparison runs 3 times.*

i. e. temp → elements[0] == val [False]

temp → elements[1] == val [False]

temp → elements[2] == val [True]

- *When **n** element is to be deleted: loop runs **n** times.*

Note : we are doing comparisons like:

Node 1 (Elements 1 through 4)

- *Target is 1st element: 1 comparison.*
- *Target is 2nd element: 2 comparisons.*
- *Target is 3rd element: 3 comparisons.*
- *Target is 4th element: 4 comparisons.*

Node 2 (Elements 5 through 8)

- *Target is 5th element: 4 comparisons (from Node 1) + 1 comparison (in Node 2) = 5 total comparisons.*
- *Target is 6th element: 4 + 2 = 6 total comparisons.*
- *Target is 7th element: 4 + 3 = 7 total comparisons.*
- *Target is 8th element: 4 + 4 = 8 total comparisons.*

Hence , here `n` elements represents total `n` elements of `N` node blocks.

***And, when `n` element is to be deleted , we actually run loop
`n` number of times .***

*If there is **n** number of elements, we can delete data at **n** positions.*

- *For position 1, the probability of deletion is $\frac{1}{n}$.*
- *For position 2, the probability of deletion is $\frac{1}{n}$.*
- *For position n, the probability of deletion is $\frac{1}{n}$.*

Hence probability of deletion will always remain $\frac{1}{n}$.

Average Time Complexity is =

probability of deletion × (number of possible iterations):

$$= \left(\frac{1}{n} \times 1 + \frac{1}{n} \times 2 + \dots + \frac{1}{n} \times n \right)$$

$$= \frac{1}{n} \times \{1 + 2 + 3 + \dots + n\}$$

$$= \frac{1}{n} \times \left\{ \sum_{i=1}^n i \right\}$$

$$= \frac{1}{n} \left\{ \frac{n(n+1)}{2} \right\}$$

$$= \frac{n^2 + n}{2n}$$

$$= \frac{n^2}{2n} + \frac{n}{2n}$$

$$= \frac{n}{2} + \frac{1}{2}$$

$$= \frac{n+1}{2}$$

Alternate simplest approach :

B. Based on Position

- ***For position 1, the probability of deletion is $\frac{1}{n}$.***
- ***For position 2, the probability of deletion is $\frac{1}{n}$.***
- ***For position n, the probability of deletion is $\frac{1}{n}$.***

Hence probability of deletion will always remain $\frac{1}{n}$ for position 1 to n:

Average case complexity:

Probability of deletion $\times$ (Summation of number of Position).

$$\begin{aligned} &= \frac{1}{n} \times \left(\sum_{i=1}^n i \right) \\ &= \frac{1}{n} \times \left(\frac{n(n+1)}{2} \right) \\ &= \frac{n+1}{2} \end{aligned}$$

A. Total average comparison cost: $c_s \times \left(\frac{n+1}{2} \right)$

B. Average Node Traversal

The unrolled linked list consists of N total nodes. Finding a randomly distributed element requires hopping through the node blocks.

When searching for an element, the outer while (temp $\neq$ NULL) loop hops from node to node.

If list has a total of N nodes, the element must be in one of them:

- If the element is in the 1st node, the loop runs 1 time.***
- If the element is in the 2nd node, the loop runs 2 times.***
- If the element is in the N – th node, the loop runs N times.***

If we assume the elements are distributed evenly across the nodes ,picking a random element means it is equally likely to be located inside any of the N nodes. Therefore, the probability of the element being in any specific node is $\frac{1}{N}$.

$$\text{Average} = (\text{Prob. of Node 1} \times 1 \text{ hop}) + (\text{Prob. of Node 2} \times 2 \text{ hops}) + \dots + (\text{Prob. of Node } N \times N \text{ hops})$$

Substitute the probability $\left(\frac{1}{N}\right)$ into the equation:

$$\text{Average Traversals} = \left(\frac{1}{N} \times 1\right) + \left(\frac{1}{N} \times 2\right) + \left(\frac{1}{N} \times 3\right) + \dots + \left(\frac{1}{N} \times N\right)$$

$$= \frac{1}{N} \times (1 + 2 + 3 + \dots + N)$$

$$= \frac{1}{N} \times \sum_{i=1}^N i$$

$$= \frac{1}{N} \times \frac{N(N+1)}{2}$$

$$= \frac{N+1}{2}$$

$$\text{B. Total Average Traversal cost: } c_n \times \left(\frac{N+1}{2}\right)$$

C. Average Array Shift

```
for (int j = i; j < temp->numElements - 1; j++) {  
    temp->elements[j] = temp->elements[j + 1];  
}
```

Assuming if a node is 100% full, then it has B number of elements:

Loop generally traverse from 0^{th} index to $(B - 2)^{th}$ index, where last shift will be element at $B - 2^{th}$ index = $B - 1^{th}$ index.

Therefore, we can compute it as follows:

*(Deleting at the very end i.e. any node's B^{th} element, sitting)
At Index $B - 1$*

There are 0 elements to the right of it. Number of shifts = 0.

*(Deleting at the second to last spot i.e. any node's $B - 1^{th}$ element, sitting)
At Index $B - 2$*

There is 1 element to the right of it. Number of shifts = 1.

*(Deleting at the third to last spot i.e. any node's $B - 2^{th}$ element, sitting)
At Index $B - 3$*

There is 2 element to the right of it. Number of shifts = 2.

.
.

*(Deleting at the very beginning i.e. any node's 1^{th} element, sitting)
At Index 0*

There is $B - 1$ element to the right of it. Number of shifts = $B - 1$.

$$\text{Average of Array Shift} = \frac{\text{Sum of all possible shift counts}}{\text{Total number of possible positions}}$$

$$\text{Average of Array Shift} = \frac{0 + 1 + 2 + \dots + B - 1}{B}$$

$$= \frac{\sum_{i=0}^{B-1} i}{B}$$

$$= \frac{0 + \sum_{i=1}^{B-1} i}{B}$$

$$= \frac{\sum_{i=1}^{B-1} i}{B}$$

$$= \frac{\frac{(B-1)((B-1)+1)}{2}}{B} \left[\begin{array}{l} \sum_{i=1}^n i = \frac{1}{2} \times n \times (n+1), \\ \text{plugging } n = B-1, \text{ we get:} \\ \frac{1}{2} \times (B-1) \times ((B-1)+1) \end{array} \right]$$

$$= \frac{\frac{(B-1)(B)}{2}}{B}$$

$$= \frac{(B-1)(B)}{2} \times \frac{1}{B}$$

$$= \frac{B-1}{2}$$

The shifting operation only ever happens exactly one time during the entire function call. Therefore for a single node's average case is required here, not for continuous N^{th} node.

C. Total Average Shifting cost: $c_{sh} \times \left(\frac{B+1}{2}\right)$

D. Deletion Bookkeeping Costs

```
temp->numElements--;

        cout << "Element " << val << " deleted
successfully.\n";

        // Check if the node is now completely empty
        if (temp->numElements == 0) {
            if (prev == NULL) {
                head = temp->next;
                free(temp);
            } else {
                prev->next = temp->next;
                free(temp);
            }
            cout << "An empty node was removed from the
list.\n";
        }
```

The actual deletion process (decrementing numElements, evaluating if conditions, and potentially freeing an empty node) executes exactly one time when the element is successfully found.

This is a fixed constant cost.

$$\therefore \text{D.Total Deletion Cost} = c_d \times 1 = c_d .$$

$$\begin{aligned} T_{avg} = & \text{Total average comparison cost: } c_s \times \left(\frac{n+1}{2}\right) \\ & + \text{Total Average Traversal cost: } c_n \times \left(\frac{N+1}{2}\right) \\ & + \text{Total Average Shifting cost: } c_{sh} \times \left(\frac{B+1}{2}\right) \\ & + \text{Total Deletion Cost} = c_d \times 1 = c_d . \end{aligned}$$

$$T_{avg} = c_s \times \left(\frac{n+1}{2}\right) + c_n \times \left(\frac{N+1}{2}\right) + c_{sh} \times \left(\frac{B+1}{2}\right) + c_d$$

To express this in standard Big – O notation, we strip away all constant multipliers (like $\frac{1}{2}$), exact hardware costs (c_n, c_s, c_{sh}, c_d), and lower – order additive constants:

$$T_{avg} = O(n) + O(N) + O(B) + O(1)$$

- **Node Capacity is Constant:** Because $B = 4$, the maximum shifting boundary $O(B)$ is just $O(4)$, which simplifies directly to $O(1)$.

- **N scales with n :** Because the total number of nodes is $N \approx \left\lceil \frac{n}{4} \right\rceil$, N is directly proportional to n .

This means traversing N nodes takes the same linear time as searching n elements, making $O(N)$ mathematically equivalent to $O(n)$.

$$\begin{aligned} \therefore \text{In terms of Elements}(n): T_{avg} &= O(n) + O(n) + O(1) + O(1) \\ &= O(n) \left[\begin{array}{l} \text{As, } n \text{ is the} \\ \text{dominating} \\ \text{Term} \end{array} \right] \end{aligned}$$

Because $O(n)$ is the term with the highest growth rate, it absorbs all constants.

$$\begin{aligned} \text{or, In terms of Nodes}(N): T_{avg} &= O(N) + O(N) + O(1) + O(1) \\ &= O(N) \left[\begin{array}{l} \text{As, } N \text{ is the} \\ \text{dominating} \\ \text{Term} \end{array} \right] \end{aligned}$$

Because $O(N)$ is the term with the highest growth rate, it absorbs all constants.

Therefore, the **final Average Case Time Complexity** is definitively:

$O(n)$ or $O(N)$.